Introduction to Garbage Collection

Christian Rinderknecht

Eszterházy Károly Katolikus Egyetem

September 2026

Garbage Collection

The runtime systems of many programming languages feature **dynamic memory management**.

The first aspect of that management is **allocation**, that is, the execution of the program can create pieces of data (called **objects** in this context) that can outlive the function that created them, even last the total running time.

Those objects exist in a memory area called the **heap**. More precisely, a **continuous block** is allocated for each object, whose size depends on the **type** of the object.

Garbage Collection

For example, in `Java`, we can declare a type for the cells in a linked list of integers as follows:

```
class Cell {
  int   content;
  Cell  next;
}
```

Each cell contains an integer and a **pointer** to another cell. The pointer is implicit in the recursive occurrence of the type name `Cell`. Here, the field `content` is a primitive type, so it is not a pointer.

A pointer is either the address of an object in memory, or a special value **nil** (noted `null` in `Java`). Consequently, each cell requires at least two **words** (one for each field and perhaps more, depending on the data layout).

Garbage Collection

A cell can be allocated with the built-in function `new`, like so:

```
Cell c = new Cell ();
```

A word is the smallest amount of memory that can be addressed and filled by a primitive type or a pointer — 64 bits in modern computers.

In other words, we can imagine memory as an array of words indexed by addresses. Those words may be interpreted as an address or a primitive type.

Then we can modify the contents of the fields, as many times as we wish to. Every call to `new` returns the **address of a new block** drawn from the available memory.

Garbage Collection

Because each allocation removes from available, finite memory, it is necessary for the GC to **free the memory used by useless objects**, which it needs to define and find.

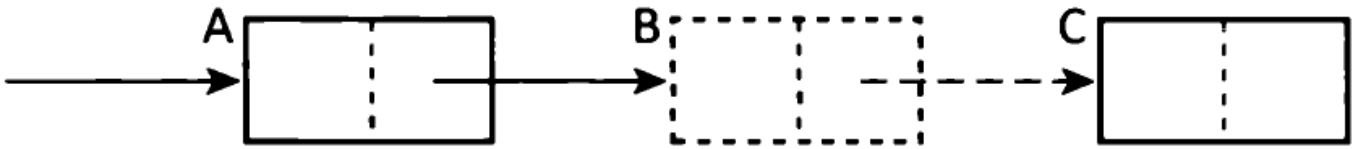
In some programming languages, like `C`, this task is entirely delegated to the programmers. They have a predefined procedure `free` that frees a memory block:

```
free(c);
```

After it is executed, the block pointed at by `c` is considered available again, so its address can be reused by a future allocation.

Garbage Collection / Memory leak

In `C`, the premature release of an object may lead to errors:



Here, `B` has been freed. The live object `A` now contains a **dangling pointer**, and the memory occupied by `C` has **leaked**: `C` is not reachable, but it cannot be freed.

Another common mistake is freeing an already freed object (like `B` above): this is an undefined behaviour in `C`, and thus a source of **non-determinism**.

Garbage Collection

In `Java`, there is no equivalent to the `free` procedure of `C` because the identifying and freeing of useless objects is left to the runtime system.

That part of the runtime is called the **garbage collector** (GC), or 'collector' in short. At regular intervals, it interrupts the program (called hereafter the **mutator**), analyses the heap, find out some or all useless objects, and frees them.

This technique was invented by McCarthy, creator of the functional programming language `Lisp` in 1960. It is widely used in mainstream languages like `Java`, `C#`, `Haskell` or `OCaml`.

Beyond avoiding the two errors above, a GC may also free many objects at a time, which is more efficient than any sequential, manual deallocation.

Garbage Collection

How does a GC determine what objects are useless, since it has no access to the source code or the programmer (interactively)?

The GC can rely on the following condition:

If, at a certain moment, a program does not have (anymore) a pointer to an object, that object is considered useless.

That condition is not true in `C`, where pointers can be forged from integers (by type casting), but it holds in `Java` for example.

Therefore, the memory block of that object can be reused for another, without the program being able to observe that reuse.

Garbage Collection / Roots & Liveness

When, at the current point in the execution of a program, the address of an object is known, that object is said to be **alive**, and **dead** otherwise.

We can define inductively the notion of liveness:

Definition 1
*Both the global and local variables of the program are called **roots**. By definition (base case), the object pointed at by a root is alive. Moreover, if an object is alive and one of its fields contains the address of another object, then that object is alive too (inductive case).*

What counts as global and local variable depends on the programming language. In `Java`, the former are the `static` fields (a.k.a. instance fields), and the latter are methods arguments (including `this`) and the variables declared in the method being executed.

Garbage Collection

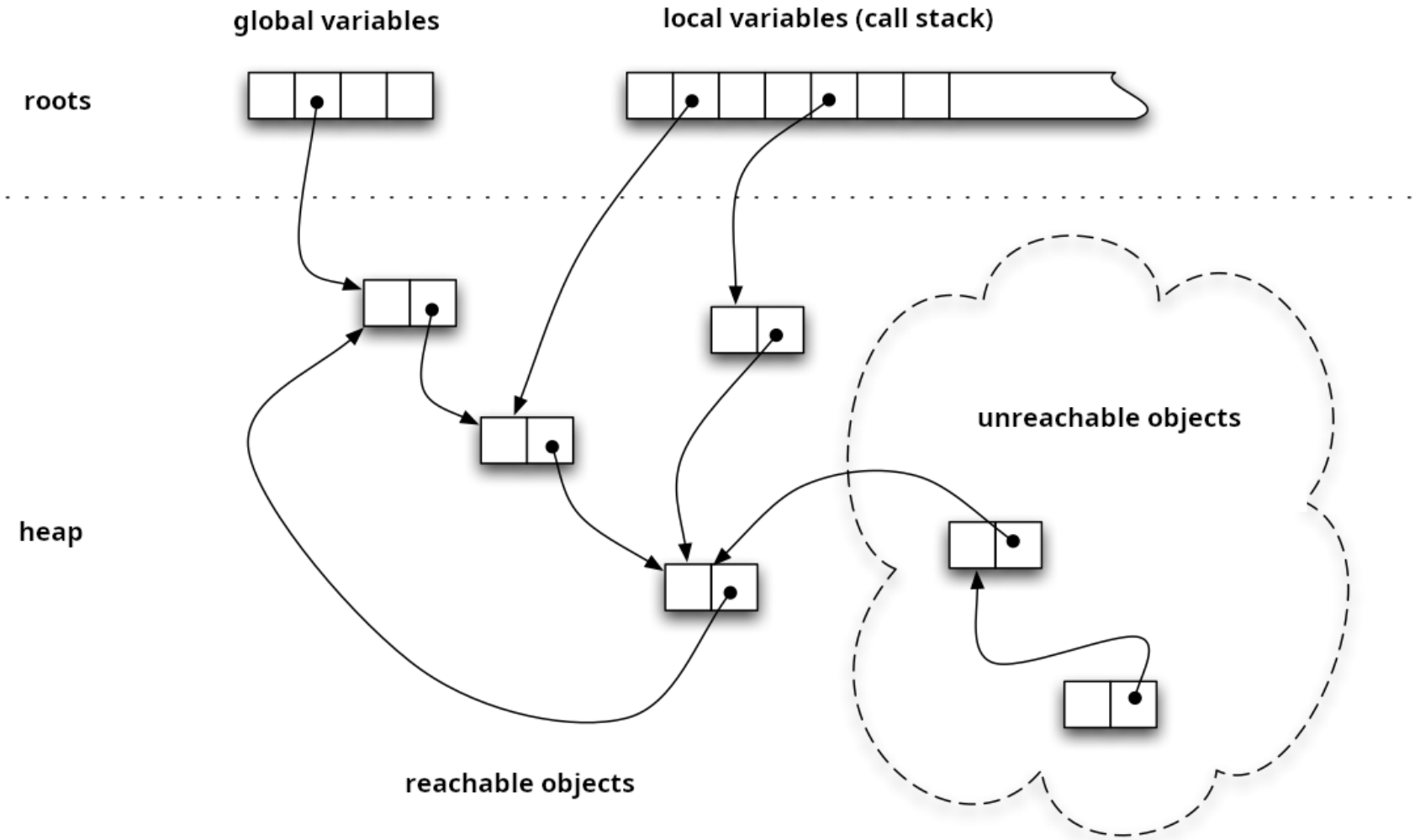
The heap can now be seen as a **graph**, where the **vertices** correspond to the roots and the objects, and the **edges** model the pointers between objects (that is, if one's block contains a pointer to another's).

Sometimes we will use interchangeably 'block' and 'object' when referring to blocks in the heap. Likewise 'pointer' and 'address'.

The live objects are then the vertices **reachable from the roots**.

Therefore, to determine the live objects at a given time, the GC has to traverse the graph of objects from the roots.

Garbage Collection



Garbage Collection

We started this introduction by referring to ‘useful objects’, but now we see that the appropriate concept is that of ‘live objects’, which is the same as ‘reachable objects’.

It is obvious that all dead objects are useless, but the reciprocal is not true: a live object might be useless if the program does nothing with it. They are allocated in the heap but cannot be reclaimed. This is a case of **memory leak**. (We saw page 6 another case of memory leak in `C`.)

Garbage Collection / Mark & Sweep

One of the simplest technique for garbage collection is known as **mark & sweep**.

As the name indicates, it consists of two passes:

1. mark: traversing the *graph* of objects from the roots while marking the visited nodes,
2. sweep: traverse the *whole* heap (the linear piece of memory, not the graph view): all the unmarked objects are freed, and the marks of the others are reset (to mean 'not visited').

Crucially, during the mark & sweep, **the mutator is stopped**.

Garbage Collection / Marking

In the following pseudo-code, we will assume that each object o has two fields, o.left and o.right, each containing a pointer to another object, or the special value `nil`. Each object also contains a **bit field** denoting the states 'marked' and 'unmarked' (default): this is part of the **metadata**.

Obviously, the bit marks from the first phase constitute the information carried over to the sweeping phase, but they also play an essential rôle during marking: they tell the traversal algorithm to not visit a vertex more than once, so the total cost of traversing the graph remains **linear in the number of vertices**.

This is crucial if some objects are shared through distinct paths, in particular if the graph contains cycles.

Garbage Collection

We stated earlier that each object would contain a bit field for the mark. This is the most common solution: put the mark into a **header**, that is, metadata about each object. The downside is that the cost per object becomes (at least) one word, even if only a bit is required and there is only one field of data (that is the worst case: the size of the metadata is the same as that of the data).

There is another choice: a separate table, called a **bitmap**, where the bits are stored in a compact manner: one word will encode many bit marks (64 on modern hardware architectures). This enables getting rid of the object headers and saving a lot of memory, especially if there are many small objects, at the expense of extra time accessing and decoding the table (mapping object addresses to bit positions and vice-versa).

Garbage Collection / Recursive Marking

For the marking of objects, any graph traversal can do. For example, let us run a **depth-first search** from each root:

```
    DFS(o)
1   if o ≠ nil and o is not marked
2       then mark o
3            DFS(o.left)
4            DFS(o.right)
```

Note how we use indentation to suggest blocks of instructions. Finally:

```
    MARK()
1   for each root r do DFS(r)
```

Garbage Collection

This program terminates because the bit marks are never set twice nor reset.

Let us count the number of calls to DFS, including the recursive calls. First, there is one call for each root. Second, two recursive calls per object being marked. Therefore, the total number of calls to DFS is $O(r+m)$, where r is the number of roots, and m is the number of marked objects.

Moreover, the flipping of the bit mark is assumed to take a constant time: $O(1)$.

Note that m only counts the live objects: if there are many dead objects, the cost does not change here.

Garbage Collection / Iterative Marking

The previous pseudo-code uses function calls, but it is better for the collector to not modify the call stack, which is meant for the mutator.

Let us write a single loop instead, one that manages a stack of pointers to be traversed.

```
    MARK()                                          ▷ Iterative definition
1   for each root r do push r
2   while the stack is not empty do
3       pop a pointer o
4       if o ≠ nil and o is unmarked
5       then mark o
6                push o.left
7                push o.right
```

Garbage Collection

We could have used a data structure other than a stack, because the order in which the objects are visited does not matter.

We also could avoid pushing **nil** pointers.

If we interpret o as a linked list so that o.left points to the contents of the cell, and o.right points to the next cell, then what is the behaviour of the code above? In particular, what is the size of the stack?

Garbage Collection

We see that *o*.left is pushed *before* *o*.right, so the next pointer to be popped is *o*.right, and so forth until the end of the list, that is *o*.right is **nil**. So the size of the stack ends up being the same as the list's. If the list is large, this might be a problem.

Instead, we could **push** *o*.left **after** *o*.right. This way, the contents of a cell would be traversed before the next cell, that is, the rest of the list. Therefore the traversal cost (in terms of memory) does not depend on the size of the list.

In a functional programming language, lists are the bread and butter of programmers, therefore a GC traversing the head before the tail of the list is a significant improvement.

Garbage Collection / Efficient Marking

```
    MARK()                                  ▷ Second redefinition, more efficient
1   for each root r do push r
2   while the stack is not empty do
3       pop a pointer o
4       if o ≠ nil and o is unmarked
5       then mark o
6                push o.right                                  ▷ Right first
7                push o.left
```

Garbage Collection / Traversal Stack

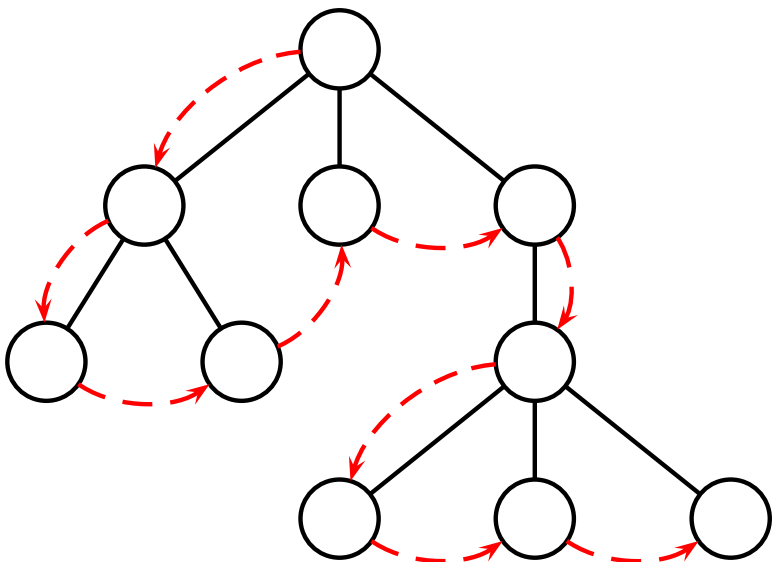
In general, an object will have more than two pointers, and the previous optimisation applies only because the compiler somehow informed the GC that this is a value of the list data type.

```
    MARK()                                   ▷ General depth-first traversal
1   for each root r do push r
2   while the stack is not empty do
3       pop a pointer o
4       if o ≠ nil and o is unmarked
5       then mark o
6                for each c in CHILDREN(o) do push c
```

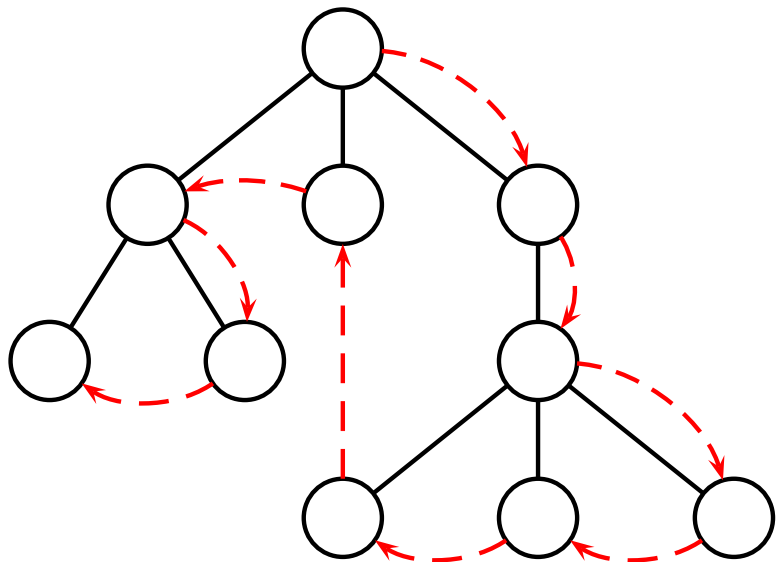
Garbage Collection / Tree traversals / Depth-first

Depending on the data structure we use to hold $\textsc{Children}(o)$, we get different traversals.

In the case of traversing a **tree**, here are two examples of **depth-first traversals**:



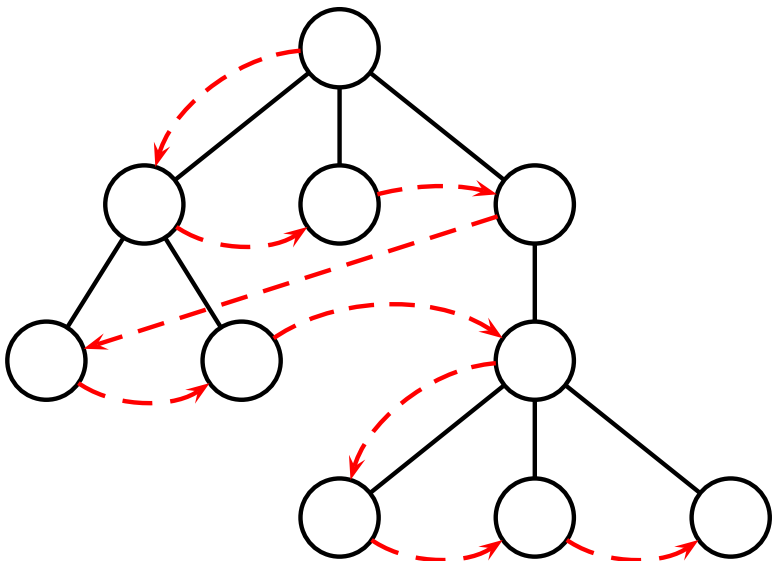
This is a rightward traversal.



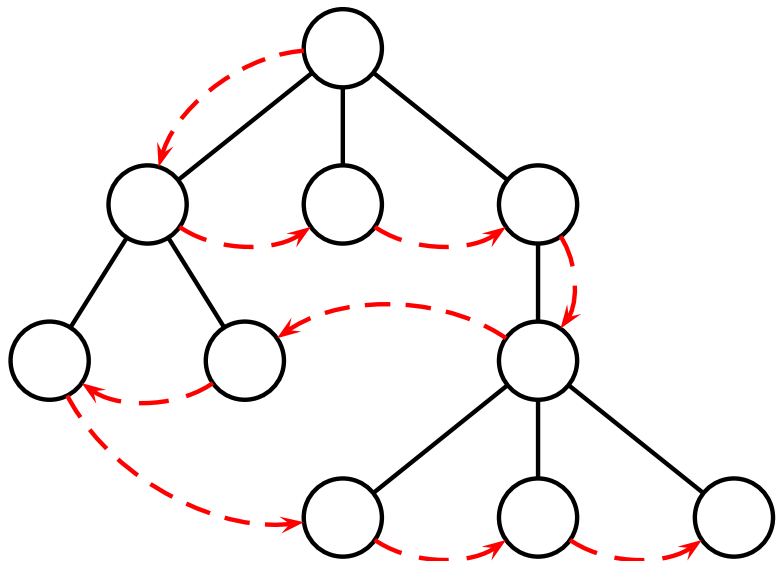
This is a leftward traversal.

Garbage Collection / Tree traversals / Breadth-first

Here are two examples of **breadth-first traversals**:



This is a rightward traversal.



This is an alternating traversal.

Garbage Collection

But how CHILDREN knows what fields of an object are pointers and which are not, for example, integers and floating-point numbers? There are several approaches.

- Like a bitmap can be used to keep track of allocated objects, the header of an object can contain a bitmap for the fields that are pointers.
- The first bit of words can be used to distinguish pointers and scalars. For example, `OCaml` uses only 63 bits to encode values, including pointers, as the leftmost bit is used to 'type' them.
- If words have a valid interpretation as pointers, we can assume they are pointers. This what **conservative GCs** do. They over-approximate live objects, so some dead objects might not be freed. This what GCs for `C` and `C++` do (those languages have no GC by default).

Garbage Collection / Traversal Stack

As we saw in the program page 22, the size of the traversal stack can be linear in the size of the reachable data. But how is that stack implemented?

It cannot be allocated in the heap with the mutator's data, as the heap must remain the same during a collection. Instead, it relies on an array whose size is fixed beforehand (static data for the collector).

An element of that array then fits in a word (interpreted as an address), which is optimal.

Nevertheless, if the heap can contain up to n objects, that array should be of size n, even if not used completely. Moreover, if each block contains at least three words (a header and two pointers), then the length of the stack-array should be a third of that of the heap, which is considerable.

Garbage Collection

One obvious solution is having **a bounded stack** that is not too large.

If it becomes full, we drop any new pointer. At the end of the marking, the collector would know that the graph was not traversed entirely (because a stack overflow sets a flag, for instance).

In that case, a linear sweep of the whole heap is performed: whenever a marked object is found to point to an unmarked object, we run the marking algorithm from it, as if it were a root. This process is iterated until a fixed point (no new object is marked): all reachable objects have then been marked.

Additionally, in case of a stack overflow, the size of the stack can be doubled for the next collection.

Garbage Collection / Pointer Reversal

Another solution by Deutsch, Schorr and Waite (DSW), more complex, consists in dispensing with the stack entirely. Instead, the pointers in the heap are modified to keep track of the path of the marking.

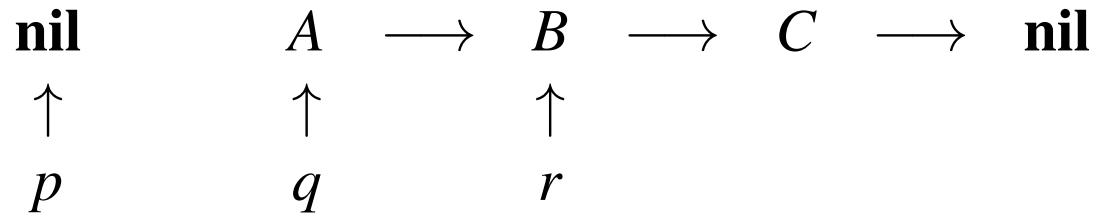
Let us illustrate the basic technique behind DSW, called **pointer reversal**, on a singly-linked list containing only primitive types, so each object o has exactly one pointer o.next. For instance

$$A \longrightarrow B \longrightarrow C \longrightarrow \textbf{nil}$$

The collector starts the traversal from A without a stack

Garbage Collection

Initially:

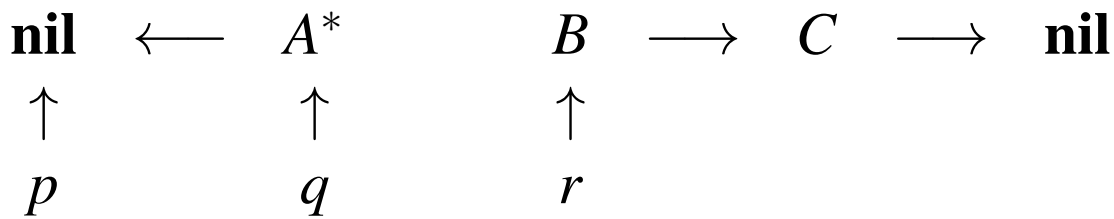


The DSW maintains three pointers:

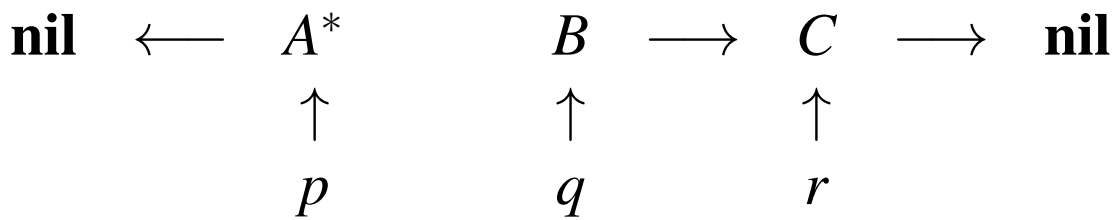
1. q points to the current node being visited (to be marked or already marked);
2. p points to the previous visited node (e.g. $p \rightarrow$ **nil** if $q \rightarrow A$);
3. r points to the rest of the list (e.g. $r \rightarrow$ **nil** if $q \rightarrow C$).

Garbage Collection

We can mark A (becomes A^*) and set A.next to point backwards to p:

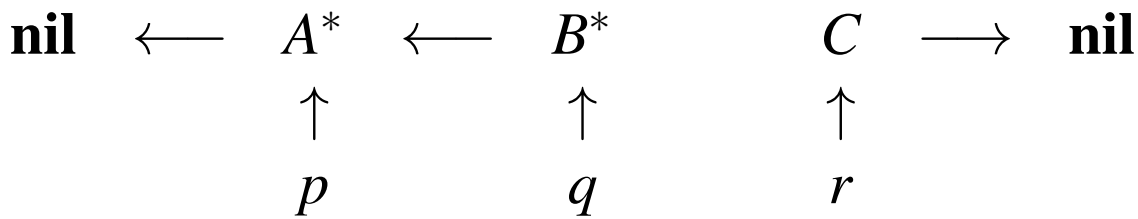


Then we move p, q and r one step to the right:

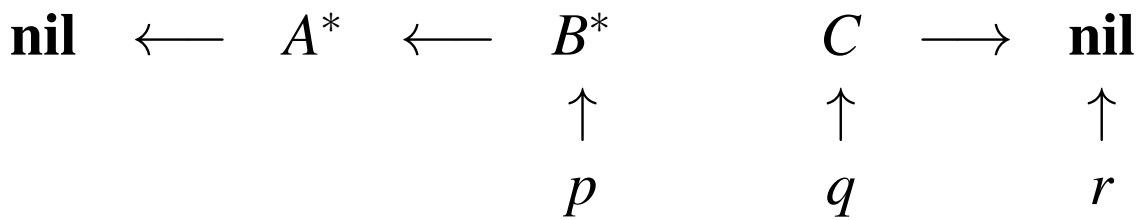


Garbage Collection

We can mark B and set B.next to point backwards to A^*:

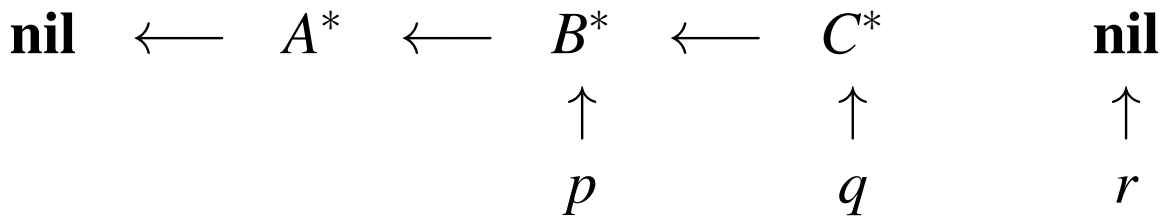


Then we move p, q and r one step to the right:

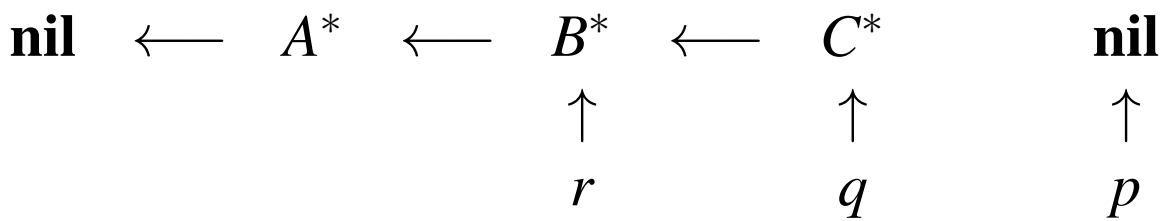


Garbage Collection

We can mark C and set C.next to point backwards to B^*:



We are done. To return, we need to reverse the pointers again using the same algorithm, except this time we start swapping p and r:



The general case (trees, graphs) is more complicated, but this gives the gist of it.

Garbage Collection / Sweeping

Let us consider now the second phase: sweeping.

During marking, we considered the graph of objects. Now we look at the heap as an array of words indexed by addresses. (This dual view of the heap, either as a graph or an array, is crucial in understanding garbage collection.)

In such a view of the heap, we see a series of blocks, either allocated to objects or free — both kinds starting with a header.

To understand how to sweep unreachable objects, we first need to know how objects were allocated first.

Garbage Collection / Allocation with Free Lists

Let us assume that the collector

- can distinguish allocated blocks from those that are free to use;
- can determine the size of each block (allocated and free);
- maintains the set of all free blocks in a **free list**.

Objects have a header. **Free blocks need a header** too:

1. to distinguish an allocated object from a free block, we store in the first word a **number unique to all free blocks**;
2. in the second word, the **size of the block**.
3. Since free and allocated blocks share the same space (no partitioning of the heap), we need to connect all the free blocks in a manner reminiscent of how the objects are interconnected in a graph. So the third word of their header contains a **pointer to the next free block**, making up the free list. The address of the first free block will be held in a global variable.

Garbage Collection

In all, free blocks require three-word headers, which is acceptable, as, for instance, we have seen that a simple linked list requires three words of data (plus the header, which is metadata).

The free list will require three main operations:

1. a predicate IsEmpty() on whether the free list is empty;
2. a procedure Allocate(o) that looks up a free block large enough to contain the object o, allocates and initialises the memory, and updates the free list (leaving perhaps a smaller free block behind), and otherwise fails;
3. a procedure Free(a) that turns the object allocated at address a into a free block that is in turn registered with the free list, perhaps after coalescing it with any contiguous ones.

Garbage Collection / Sweeping

Assuming a free list and the previous primitive operations, we can return to the second phase of the collector: sweeping.

```
     SWEEP()
1    a ← start of the heap          ▷ There is always a block at address a
2    while a < end of the heap do                ▷ Increasing addresses
3        n ← SIZE(a)                             ▷ Includes the header
4        if a is an allocated object o           ▷ All blocks have a header
5        then if o is marked                                ▷ o is alive
6                then remove the mark                ▷ Write the header
7                else FREE(a)                                ▷ o is dead
8        a ← a + n
```

Note how we call SIZE(a) *before* calling FREE(a), as the latter could destroy the information needed by SIZE, by coalescing the newly freed block with another.

Garbage Collection / Freeing blocks

One simple strategy for freeing a block is to change the header of the block and insert it at the front of the free list, like so:

```
    FREE(a)                                   ▷ Inserting at the front
1   FREEHEADER(a)                             ▷ Converting allocated header to free
2   NEXT(a), free ← free, a
```

The downside is that we do not, in general, maintain a free list whose blocks are sorted, and sortedness improves, in average, the cost of insertion.

```
    FREE(a)                                   ▷ Keeping increasing addresses
1   previous, current ← nil, free
2   while current ≠ nil and current < a do
3       previous, current ← current, NEXT(current)
4   FREEHEADER(a)
5   NEXT(a) ← current
6   if previous = nil then free ← a           ▷ Inserting at the front
7       else NEXT(previous) ← a
```

Garbage Collection / Coalescing free blocks

Sortedness enables to efficiently **coalesce** adjacent free blocks after the new block is added:

```
     FREE(a)                                   ▷ Coalescing adjacent blocks
 1   previous, current ← nil, free
 2   while current ≠ nil and current < a do
 3       previous, current ← current, NEXT(current)
 4   if previous ≠ nil and previous + SIZE(previous) = a then
 5       SIZE(previous), a ← SIZE(previous) + SIZE(a), previous
 6   else                                      ▷ No coalescing with previous block
 7       FREEHEADER(a)
 8       NEXT(a) ← current
 9       (if previous = nil then free else NEXT(previous)) ← a
10   if current ≠ nil and a + SIZE(a) = current then
11       SIZE(a), NEXT(a) ← SIZE(a) + SIZE(current), NEXT(current)
```

Garbage Collection / Allocation

We can now write the GC procedure NEW that is called by the runtime system:

```
    NEW(o)
1   if ISEMPTY()                          ▷ Checking the free list
2      then MARK()                        ▷ Marking live objects
3           SWEEP()                       ▷ Reclaiming dead objects
4           if ISEMPTY()                  ▷ Checking again
5              then NEW ← nil             ▷ Memory is really full
6              else NEW ← ALLOCATE(o)     ▷ See next line
7      else NEW ← ALLOCATE(o)             ▷ Fails if o doesn't fit
```

Garbage Collection / First-fit Allocation

Several implementations of ALLOCATE are possible. Perhaps the simplest is **first-fit allocation**, whereby the first block in the free list that can fit the object is used.

Let NEXT(p) be the address of the free block immediately after the free block p. It is an l-value, and SIZE(p) too. The value SIZE(o) is the size of o as recorded in its header. Let **free** be the head of the free list.

```
    FIRSTFIT(o)                          ▷ Use in stead of ALLOCATE in NEW
1   previous, current ← nil, free
2   while current ≠ nil and SIZE(current) < SIZE(o) do
3       previous, current ← current, NEXT(current)
4   if current ≠ nil then UPDATE(previous, current, o)
5   FIRSTFIT ← current
```

Garbage Collection

Let us use the beginning of a free block to allocate an object:

```
     UPDATE(previous, current, o)          ▷ SIZE(current) ⩾ SIZE(o)
1    if SIZE(current) = SIZE(o)
2       then NEXT(previous) ← NEXT(current)      ▷ Drop the block
3       else rest ← current + SIZE(o)            ▷ Header size managed
4            NEXT(rest) ← NEXT(current)
5            SIZE(rest) ← SIZE(current) − SIZE(o)
6            NEXT(previous) ← rest
7    WRITE(current, o)
```

Garbage Collection / First-fit Allocation (variant)

We could instead use the end of the free block:

```
    UPDATE(previous, current, o)          ▷ SIZE(current) ⩾ SIZE(o)
1   if SIZE(current) = SIZE(o)
2       then NEXT(previous) ← NEXT(current)
3            UPDATE ← current
4       else SIZE(current) ← SIZE(current) − SIZE(o)
5            UPDATE ← current + SIZE(current)
6   WRITE(UPDATE, o)
```

Note how UPDATE must be a function: it returns the address of the allocated block, and so FIRSTFIT should be rewritten slightly.

How to fuse the remaining free block, if any, with a contiguous one (lower or higher, depending where the object is allocated)? This is called **coalescence**, which reduces fragmentation.

Garbage Collection / Next-fit Allocation

First-fit allocation tends to accumulate small free blocks at the beginning of the free list, slowing further allocations.

To avoid this, **next-fit allocation** starts searching a free block where the last allocation succeeded. When the end is reached, search resumes at the beginning, hence it is sometimes called *circular next-fit allocation*.

One drawback is that objects from different phases of the mutator execution become mixed together, and, because they die at different times, this increases fragmentation.

Garbage Collection / Best-fit Allocation

Best-fit allocation finds the first smallest free block that fits the object, thus not splitting large blocks that would otherwise fit future, larger objects.

```
    BESTFIT(o)
1   bestPrev, best, bestSize, previous, current ← nil, nil, ∞, nil, free
2   while current ≠ nil do
3       if SIZE(o) ⩽ SIZE(current) < bestSize
4       then bestPrev, best, bestSize ← previous, current, SIZE(current)
5       previous, current ← current, NEXT(current)
6   if best = nil
7       then BESTFIT ← nil
8       else  BESTFIT ← UPDATE(bestPrev, best, o)
```

The loop always traverses the free list entirely, whereas FIRSTFIT could stop at the head of the list.

Garbage Collection / Drawbacks of Free Lists

When allocating objects of different sizes, the heap will contain holes (free blocks) of varying sizes: this is **fragmentation**.

1. This may lead to an **allocation failure** because an object can fit in none of those, even if the total available memory is large enough.
2. Another unpleasant effect of fragmentation is when two successive allocation requests are satisfied by free blocks that are very far apart in the free list: this breaks data locality, which incurs **cache misses** and virtual page loads.
3. Yet another drawback of a free list is simply its linear structure, yielding an allocation to be performed in **linear time** in the worst case.

Garbage Collection / Improving Free Lists

In a first-fit context, a special case of Cartesian trees called *treaps* are useful to speed up search, as they are totally ordered by addresses (like a BST) and partially by sizes (like a heap).

In a best-fit allocation strategy, the whole free list is always traversed.

We could speed up the search by having the free blocks in a **balanced search tree** ordered by sizes (as keys), for example an AVL tree or a red-black tree (RBT). This yields an $O(\log n)$ *worst case* cost, where n is the number of available blocks.

In passing, it is then better to call this structure a **free pool**.

Garbage Collection / Improving Free Lists

The height invariant of AVL trees is tricky to get right, and deletion in RBTs is also difficult.

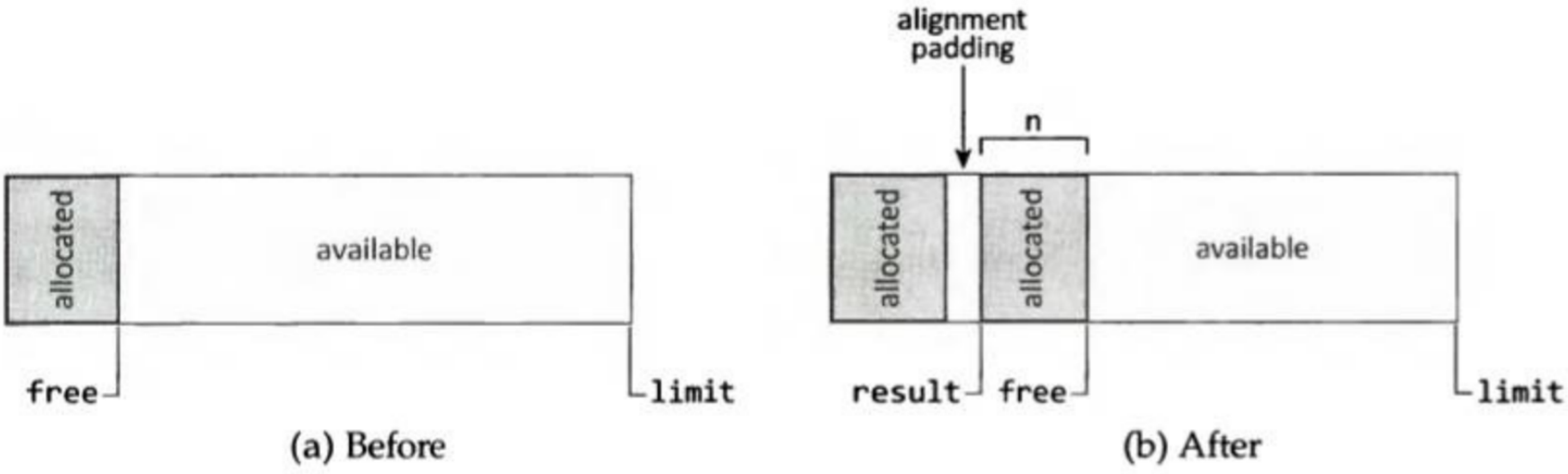
For simpler implementations, but only with a logarithmic *average cost*, **binary search trees** (BSTs) are a good compromise, or splay trees.

By far, the binary search trees are the simplest to implement.

It then saves time to enter only blocks of each size in the BST, and to chain from that node the other blocks of same size: this speeds up search, but also yields less rotations after a deletion.

Garbage Collection / Sequential Allocation

A completely different way to address the locality and linearity issues is **sequential allocation** whereby **the heap is partitioned in two** areas: one for the objects (dead or alive) and a **free zone adjacent to it**. The GC has a pointer to the start of the free zone, and another to the upper limit of that zone (to avoid overwriting in case of overflow). A new object of size n is allocated at that address (modulo some alignment padding) as follows:



Garbage Collection / Pointer Alignment

To illustrate pointer alignment, consider a word size of 4 bytes in C:

```
struct Example {
    char  a;   // 1 byte
    int   b;   // 4 bytes = 1 word
    char  c;   // 1 byte
};
```

A value of this type does **not** occupy 6 bytes, but 12:

- a starts at offset 0 (1 byte used);
- 3 bytes of padding are inserted so b starts at offset 4 (the closest multiple of 4);
- b starts at offset 4 (4 bytes uses);
- c starts at offset 8 (1 byte used);
- 3 bytes of padding are added at the end so the whole structure is a multiple of 4 bytes (word size), which is needed for uniform indexing of arrays of values of type Example.

Garbage Collection / Sequential Allocation

We assume that the collector

- can determine the size of a block by means of SIZE,
- and manages a pointer **free** to the start of the free zone, and another, **limit**, to the upper limit of that zone.

```
    SEQALLOC(o)                                  ▷ o is an object
1   free' ← free + SIZE(o)                       ▷ Read the header of o
2   if free' > limit
3      then SEQALLOC ← nil                       ▷ Insufficient memory
4      else WRITE(o, free)                       ▷ At the address free
5           free ← free'                         ▷ Bump pointer
6           SEQALLOC ← free
```

No linear search. Locality enables good cache performance. But, if objects are never moved, deallocations will create free blocks that cannot be reused, simply because they are not in the free zone: we still have the core problem of **fragmentation**.

Garbage Collection / Copying collectors

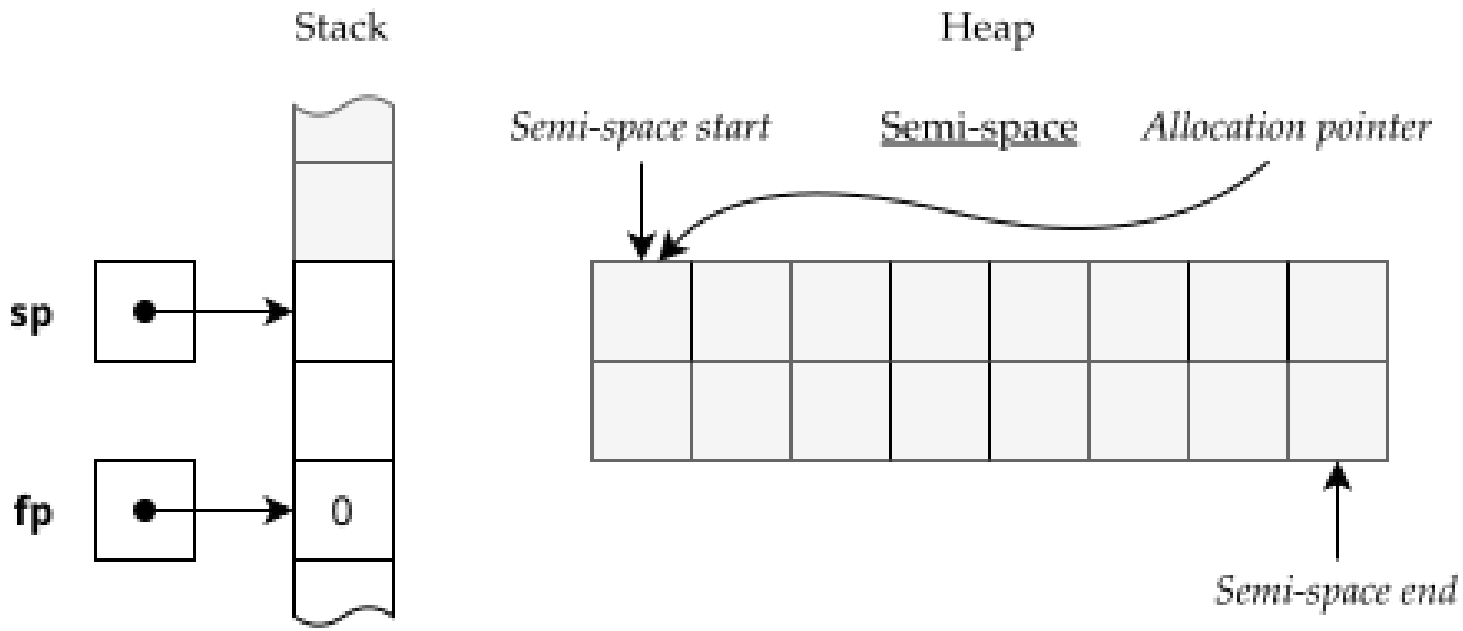
There is a class of collectors that **move/copy objects** after they have been allocated, in particular to move them next to each other. This is called **compaction**, and it gets rid of fragmentation until the next cycle.

The simplest of copying collectors is Cheney's algorithm.

The general idea is simple enough. The heap is divided in two spaces of same size: one where allocation occurs, one free. Whenever the former is full, the live objects are scanned and copied next to each other at the start of the latter. Then the rôle of the two semispaces is reversed, in particular, dead objects are not collected individually: the whole semispace where they are is now *considered* free.

Care must be taken to preserve sharing when copying the objects.

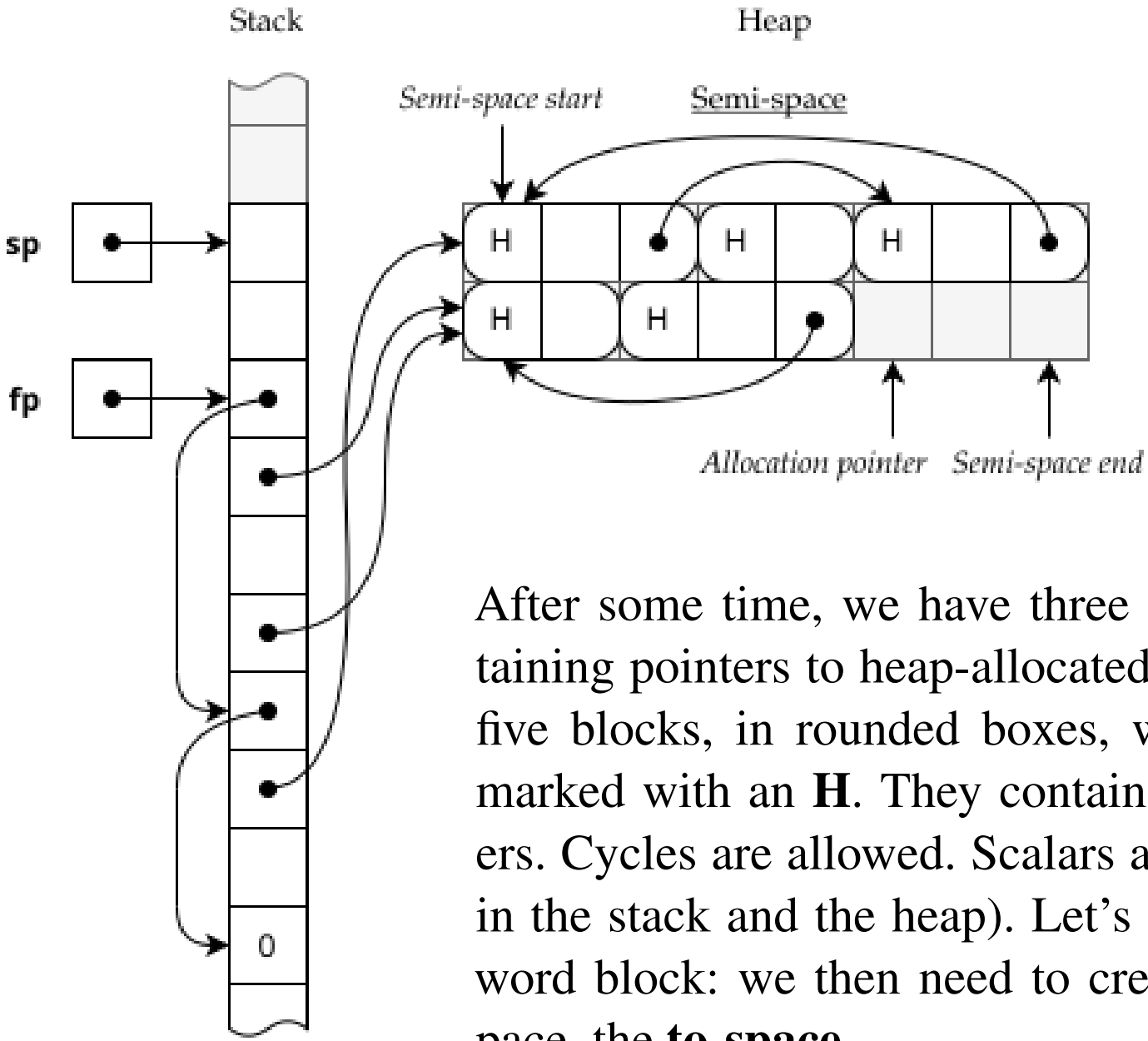
Garbage Collection / Cheney's collector



The registers **sp** and **fp** are the **stack pointer** and the **frame pointer**, respectively. (Between them are the local variables of the current function.) The **0** marks the bottom of the call stack. Frame pointers are pushed onto the stack when a function is called, therefore chaining the frames.

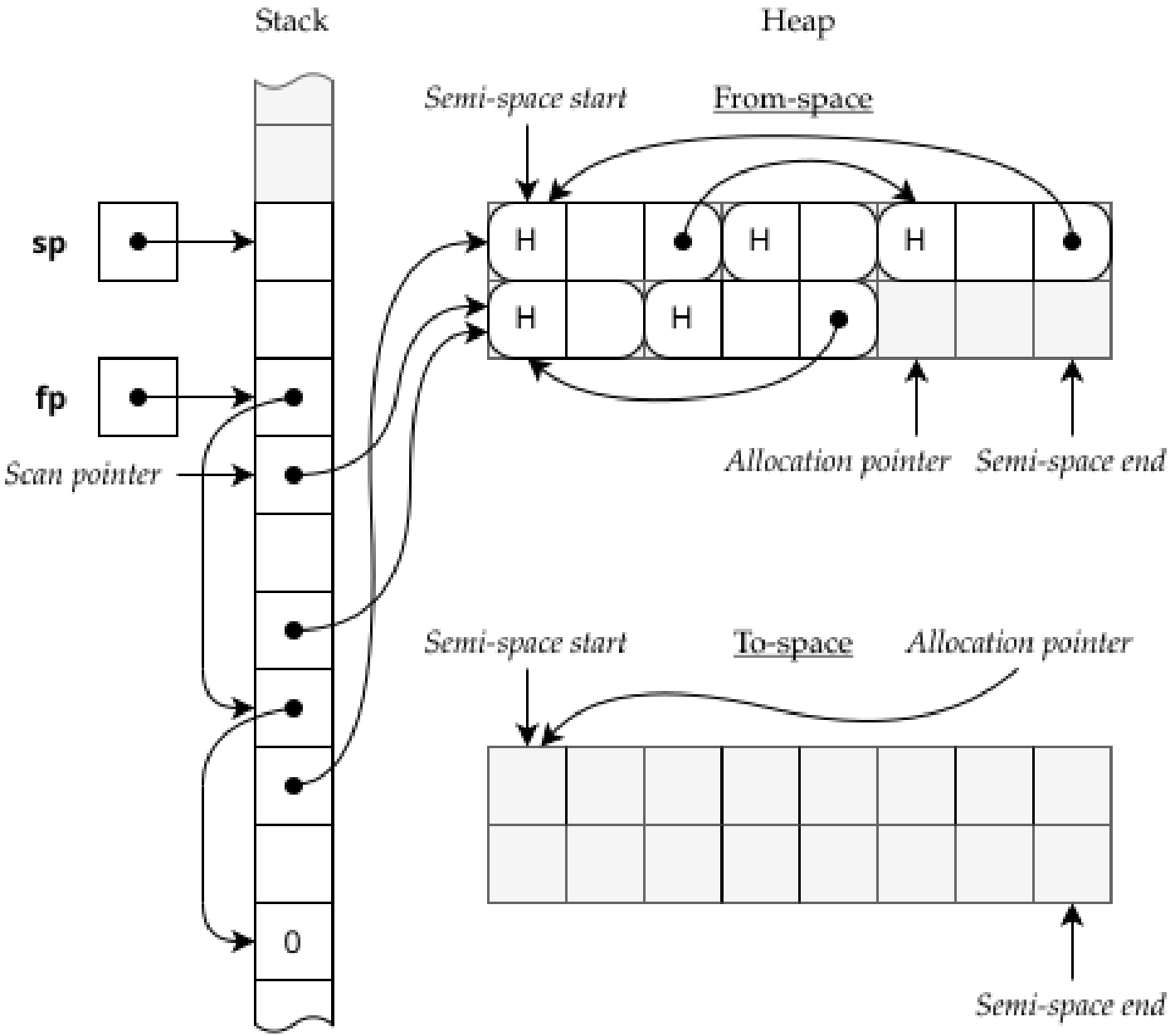
(Graphics by Vladimir Keleshev.)

Garbage Collection / Cheney's collector



After some time, we have three stack frames, containing pointers to heap-allocated blocks. There are five blocks, in rounded boxes, whose headers are marked with an **H**. They contain scalars and pointers. Cycles are allowed. Scalars are left blank (both in the stack and the heap). Let's try to allocate a 4-word block: we then need to create another semi-space, the **to-space**.

Garbage Collection / Cheney's collector



Garbage Collection / Cheney's collector

The original semispace is named **from-space** to distinguish it from the newly allocated **to-space**, where copies are created from the latter.

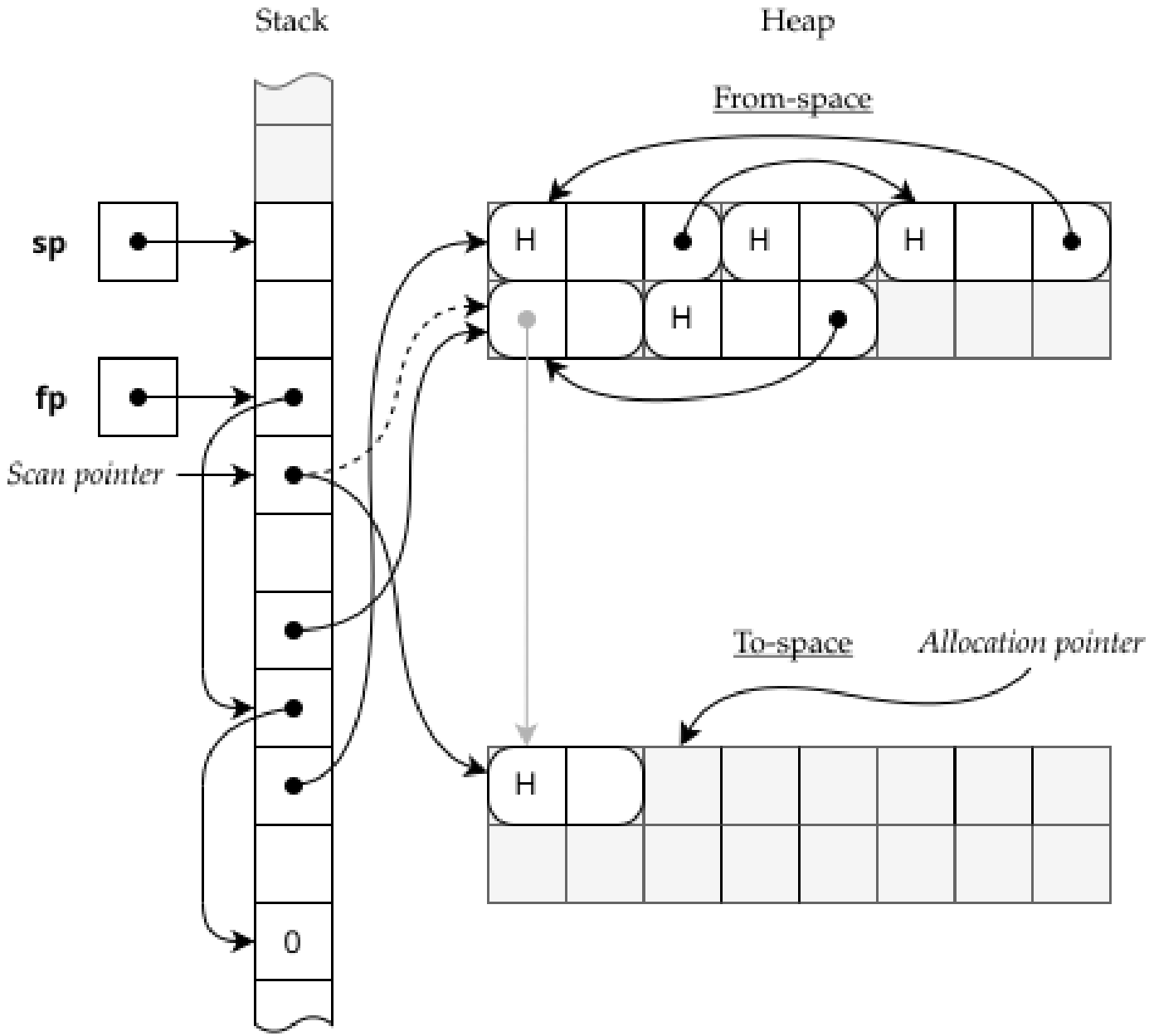
There is a **scan pointer** in the call stack that scans (downwardly) the words in the current frame, skipping scalars. We copy the block pointed by the first pointer, and bump the **allocation pointer** in the to-space.

The pointer in the stack is then updated with the copy. (Previous value with a dashed line.)

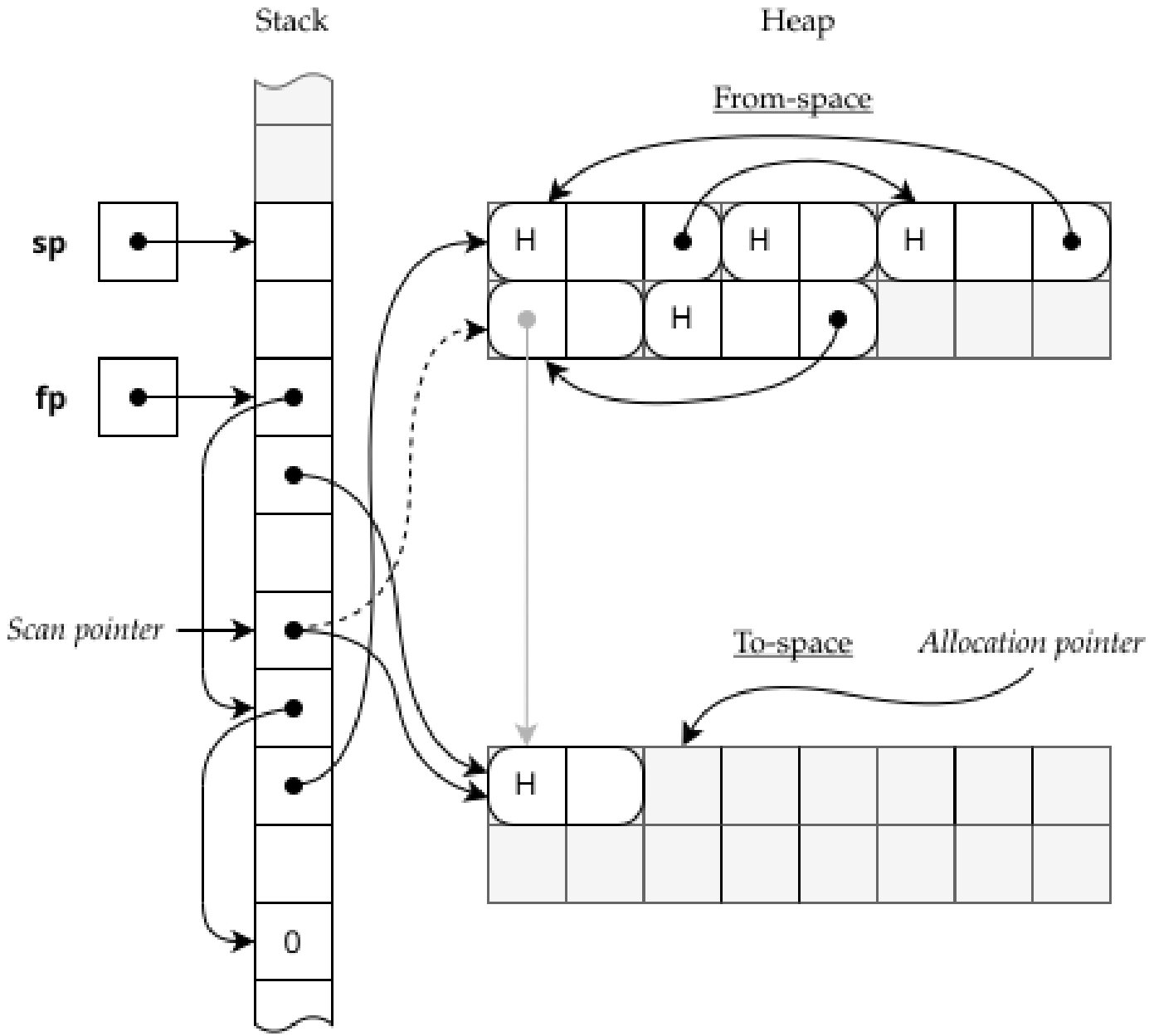
The original block header is overwritten with a pointer to the copy: this is the **forwarding pointer** (in grey). Forwarding pointers are used to preserve sharing during copying.

The last pointer of the frame goes to the already copied block, and is updated with the forwarding pointer.

Garbage Collection / Cheney's collector



Garbage Collection / Cheney's collector



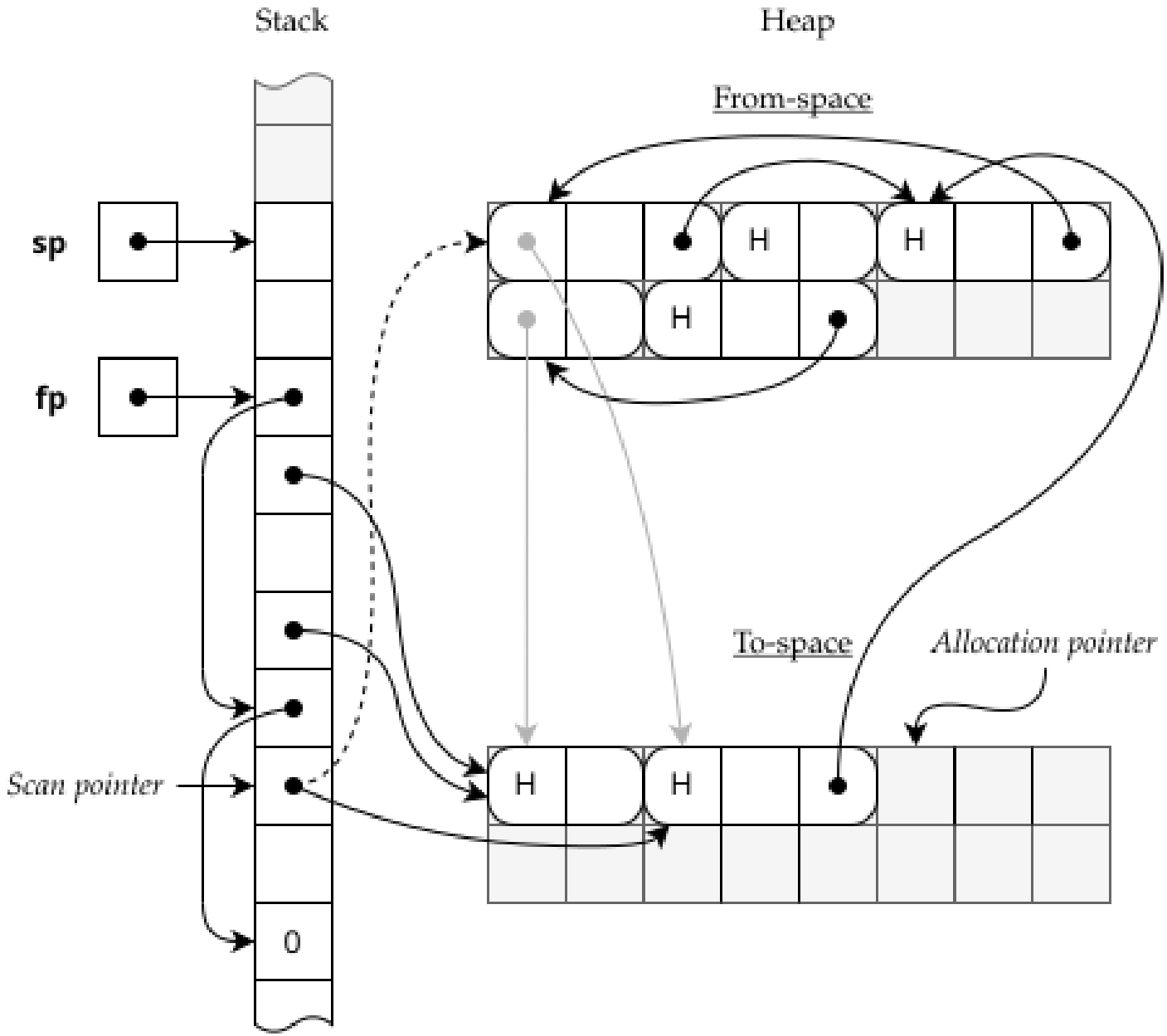
Garbage Collection / Cheney's collector

We then start scanning the next (and last) frame.

We find a pointer to the from-space. We copy it, install the forwarding pointer. Note how the copy has a pointer to the from-space.

Finally, the allocation pointer is bumped again.

Garbage Collection / Cheney's collector



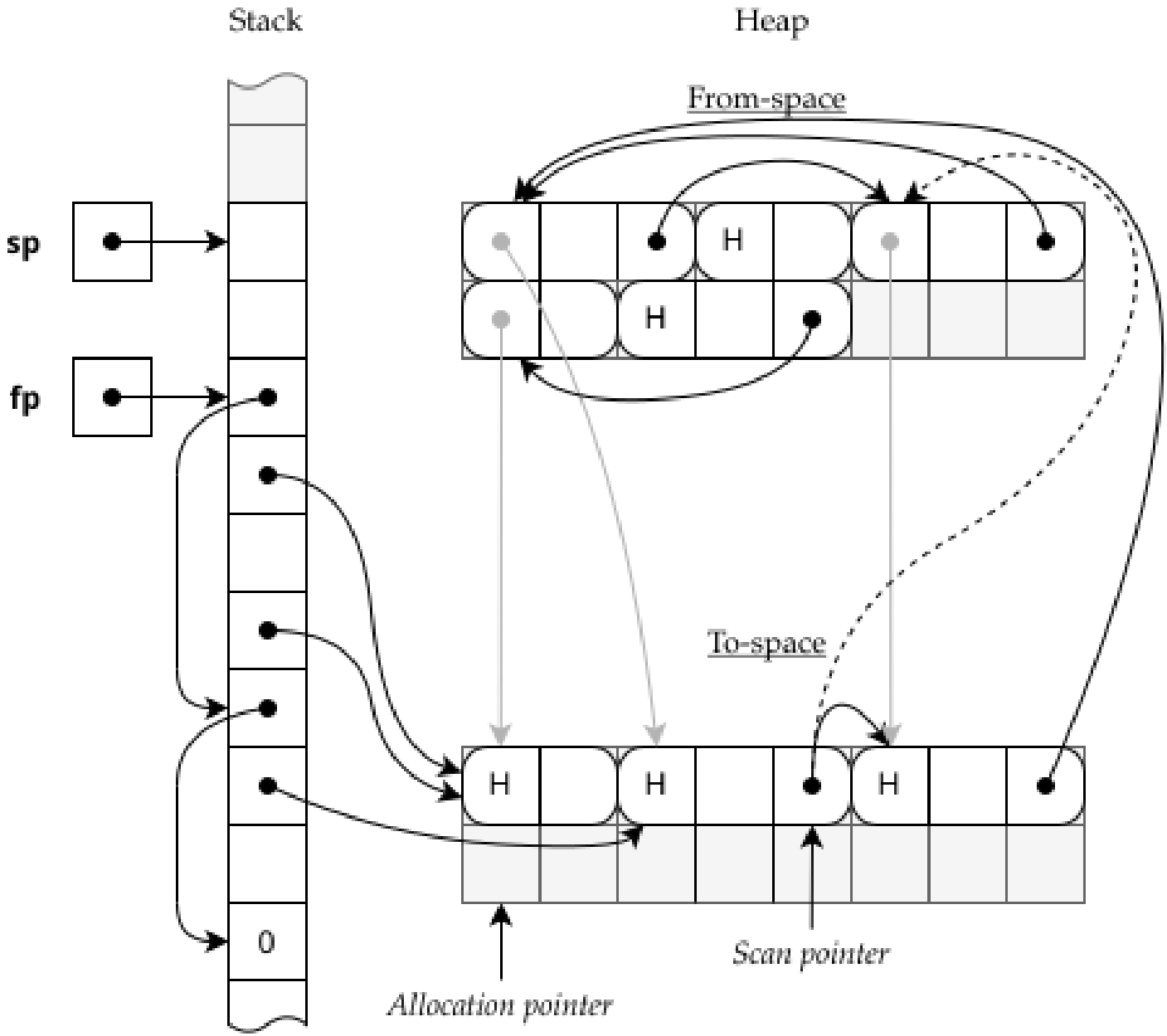
Garbage Collection / Cheney's collector

We have scanned the whole stack (arrived at the special word **0**).

Two remarks:

1. There are other roots to scan: registers and global variables (static memory). Solution: registers can be pushed on the stack before the scanning process.
2. We are not finished! There is still a block in the to-space that points to the from-space. Just like we scanned the stack, we scan the newly copied blocks in the to-space semispace (the space from the scan pointer to the allocation pointer).

Garbage Collection / Cheney's collector



Garbage Collection / Cheney's collector

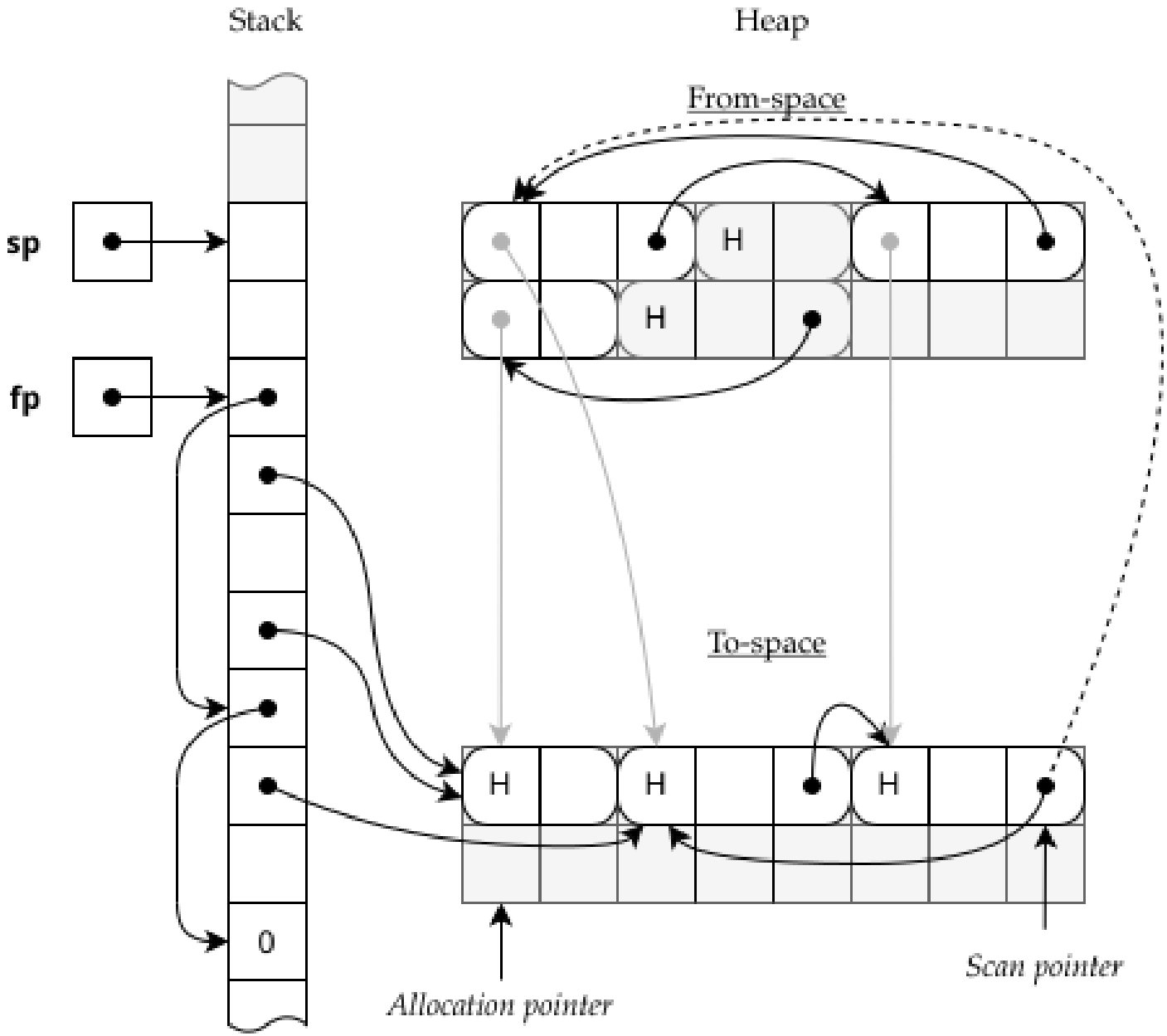
The to-space grew by one block (the third one).

We need to scan it, as it contains a pointer to the from-space. The pointed block has a forwarding pointer, which we use to update the copied block. (The original pointer is now a dashed line.)

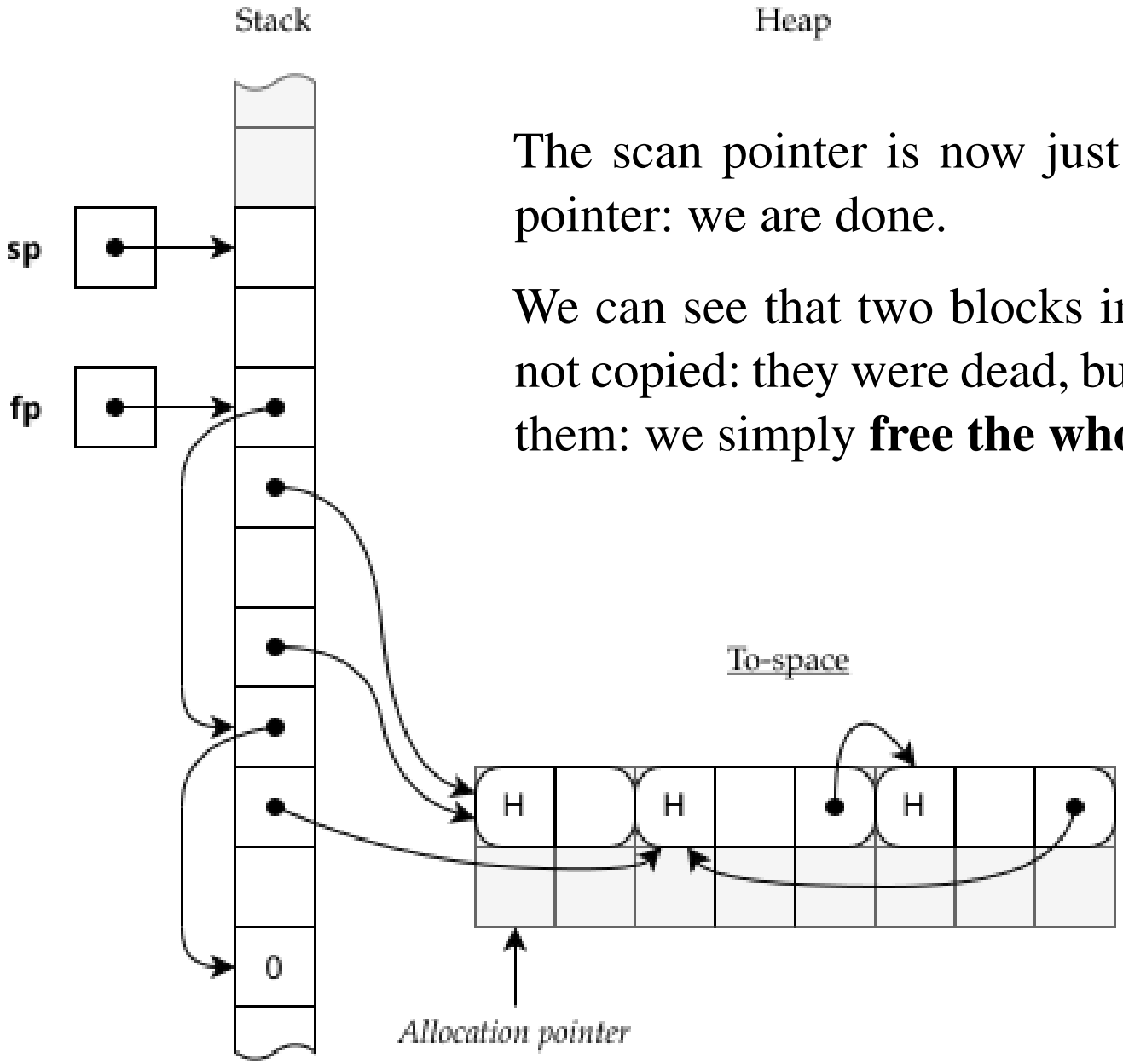
We already can see that Cheney's algorithm performs a **breadth-first traversal of the live objects**, the tell tale being the space between the scan pointer and the allocation pointer being used as a queue of objects to traverse. (These are called **grey objects**, whereas **black objects** have been visited, as well as their descendants, and **white objects** are not visited, and, in the end, dead.)

Breadth-first breaks some locality, as parents, children and grand-children objects find themselves compacted far away from each other if they have lots of siblings.

Garbage Collection / Cheney's collector



Garbage Collection / Cheney's collector



The scan pointer is now just behind the allocation pointer: we are done.

We can see that two blocks in the from-space were not copied: they were dead, but we don't need to free them: we simply **free the whole from-space**.

Garbage Collection / Cheney's collector

```
    INIT()
1   free ← fromSpace

    NEW(o)
1   free′ ← free + SIZE(o)
2   if free′ > topFromSpace then COMPACT()
3   if free′ > topFromSpace
4       then NEW ← nil
5       else WRITE(free, o)
6            NEW ← free
7            free ← free′
```

Garbage Collection / Cheney's collector

```
    COMPACT()
1   scan ← toSpace
2   free ← toSpace                                  ▷ Copy to the to-space
3   for each root r do r ← COPY(r)                  ▷ free is bumped
4   while scan < free do
5       for each p in CHILDREN(scan) do p ← COPY(p)
6       scan ← scan + SIZE(scan)
7   fromSpace, toSpace ← toSpace, fromSpace
8   topFromSpace, topToSpace ← topToSpace, topFromSpace
```

Garbage Collection / Cheney's collector

```
    COPY(p)
1   if FORWARDED(p)
2       then COPY ← FORWARDING(p)
3       else MOVE(p, free)                          ▷ Cannot fail
4            FORWARDING(p) ← free
5            COPY ← free
6            free ← free + SIZE(p)
```

Garbage Collection / Generational collector

It was observed that new objects die young. Moreover, in functional languages, those objects tend to be small (about three words of contents).

Therefore we'd like to scan new blocks more frequently, and scan older blocks more rarely. This distinction, young versus old objects, each allocated in two separate spaces of the heap, gives rise to **generational collectors**.

Each space, called a generation, can use their own collecting algorithm, as long as, after the collection of the young generation, the live young objects are promoted into the old generation.

Cheney's algorithm is a good choice for the young generation, because its cost is proportional to the number of live objects: if a lot of objects die young, the generation can be small and compaction cost remain low.

Garbage Collection

There are a lot more GC algorithms that we cannot cover, like

- **incremental collectors** that do not stop the mutator to completely collect the heap: instead they partially collect what they can in a short cycle, making sure that, in the long run no dead object leaks (this is good for soft real-time and interactive mutators, like games);
- **concurrent collectors** that do not stop the mutator at all, and even can handle multiple mutators sharing the same heap (think multicore architectures);
- **distributed collectors** that work when there are heaps managed on distinct physical computers, each running one or many mutators.